

From Navigation to Intention: Reframing the Web Experience through Goal-Driven Interfaces

Luca Cordioli
luca.cordioli@polimi.it
Politecnico di Milano

Department of Electronics, Information and
Bioengineering
Milan, Italy

Maristella Matera
maristella.matera@polimi.it
Politecnico di Milano

Department of Electronics, Information and
Bioengineering
Milan, Italy

Abstract

The dominant paradigm for interacting with the Web has long relied on menu-driven navigation and complex graphical structures that can impose a high cognitive load on users. Recent advances in Large Language Models (LLMs) enable flexible NL understanding, yet purely conversational systems lack the stability and visual affordances required for complex tasks. This paper introduces *Mirage*, a framework that bridges these paradigms by making graphical user interface (GUI) components interpretable and orchestrable by LLMs, enabling on-the-fly construction of simplified UIs tailored to user requests. Developers declare application functions as reusable components with semantic descriptions and typed schemas; *Mirage* maps user goals, expressed in natural language, to the most relevant components and renders them as interactive interfaces. We implemented *Mirage* as a Next.js framework and validated it in a user study comparing the new paradigm to the navigation of a traditional website. Results show that *Mirage* significantly reduces task completion time, lowers cognitive workload, and achieves higher usability without compromising accuracy. Our approach also highlights new design directions toward supporting on-the-fly building of goal-driven interfaces.

CCS Concepts

• **Software and its engineering** → **Software design engineering**; • **Human-centered computing** → **User interface design**; **Systems and tools for interaction design**.

Keywords

Goal-driven interfaces, Large Language Models (LLMs), Component-based frameworks, Conversational interaction, Usability evaluation

ACM Reference Format:

Luca Cordioli and Maristella Matera. 2026. From Navigation to Intention: Reframing the Web Experience through Goal-Driven Interfaces. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792479>



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792479>

1 Introduction

The dominant paradigm for interacting with complex web applications has remained largely unchanged for decades, relying on graphical user interfaces (GUIs) structured around menus, hierarchical navigation, and intricate forms. The growing complexity of applications under the WIMP (Windows, Icons, Menus, Pointer) model can impose a significant cognitive load on users [28, 40, 49]: navigating deep sub-menus or filling out multi-field forms for routine tasks increases the cognitive burden, forcing users to learn the specific layout and logic of each interface rather than focusing on their primary objective [40]. This effort is further amplified when interacting on small-screen devices [18, 49], and for people experiencing permanent or situational impairments [37].

In response to these limitations, conversational user interfaces, particularly voice-based agents, have gained widespread adoption. Historically, their effectiveness in complex scenarios was hampered by a reliance on rigid intent-and-slot-filling architectures, which demonstrated brittleness when handling out-of-scope requests or nuanced user language [21]. Large Language Models (LLMs) have reshaped this landscape, thanks to sophisticated zero-shot reasoning, enabling the interpretation with unprecedented accuracy of multi-step and even poorly phrased user commands [5, 33].

However, while LLMs largely solve the problem of language interpretation, they do not resolve the limitations of a purely conversational modality. For data exploration, comparative analysis, or high-fidelity visual feedback, the principles of direct manipulation and visual information seeking [41] remain fundamental [25]. This creates a new dichotomy: the powerful language understanding of LLMs versus the precision and stability of GUIs.

To bridge this gap, this paper introduces *Mirage*, a novel framework enabling the creation of web interfaces where GUI components, built on a declarative framework like Next.js, are interpretable and orchestrable by LLMs. The core principle is that developers create and annotate their components with semantic metadata describing their functions, required inputs, and possible constraints. An LLM then acts as an interpreter, mapping a user's NL intent to the most appropriate component and pre-filling it with the data extracted from the intent. In line with recent trends¹, this approach transforms the interaction paradigm from being "menu-driven" to "language-driven," amplifying and combining the benefits deriving from the expressiveness of NL for goal expression, and the clarity and control of a GUI for rendering data and functions responding to

¹On Oct 6th 2025, OpenAI introduced apps in ChatGPT and a new Apps SDK, enabling third-party, interactive apps to run directly inside the chat [32].

the users' needs. Beyond single-application workflows, the declarative nature of Mirage components enables them to be composed across domains.

By unifying declarative UI patterns [10, 34, 50] with the reasoning capabilities of modern LLMs [46], the approach discussed in this paper offers a path toward the definition of a unified conversation-driven framework that allows users to interact with a central goal-based, conversational interface, and then orchestrates heterogeneous components accommodating diverse usage situations. The main contributions of this work, therefore, are:

- We discuss how the new LLM capabilities prompt a rethinking of how users access and interact with the Web, suggesting the need for novel paradigms.
- We outline the design principles (declarativity, reusability, interpretability, modality blending) that guided the Mirage design, and the architectural workflow that translates NL utterances into rendered, actionable UI elements.
- We report the results of a comparative user study that evaluates Mirage against a traditional web interface in a realistic classroom booking scenario within a university intranet.
- We discuss the implications of our findings for the future design of natural language, goal-oriented interfaces for accessing the Web, proposing a new interaction paradigm that integrates NL intent expression and situational graphical rendering of data and functions.

The paper is organized as follows. Section 2 reviews prior work on conversational interfaces, component-based UI development, and LLM-driven orchestration. Section 3 introduces guiding design principles and details the Mirage architecture, component model, and interaction workflow. Section 4 presents our user study, including its design and quantitative and qualitative findings, while Section 5 reports on the technical evaluation of the framework. Section 6 then discusses implications and outlines limitations and directions for future research. Finally, Section 7 concludes the paper.

2 Background and Related Work

Despite the evolution of interactive technologies and related interaction paradigms, many foundational approaches to human–computer interaction, such as hypertext navigation, command-based interaction, and linguistic interfaces, continue to inform how users access functionality on the Web. Within this landscape, the predominant interaction model on the Web remains rooted in hierarchical structures built around nested menus and multi-page forms [20, 40, 48].

While effective for well-structured and limited information spaces, this navigation paradigm faces significant scalability challenges documented in the literature since the early Web [19, 31]. As the number of features in an application grows, interfaces become cluttered, and navigation paths become deeper and more complex. These issues are exacerbated on emerging touchpoints that constrain display and stepwise exploration, e.g., voice-first assistants, wearables, in-vehicle systems, and small-screen devices [2, 6, 27], and are further amplified for users with permanent or situational impairments [37]. Meanwhile, AI-mediated interaction paradigms (conversational, intent-based) are conditioning users to expect immediate, goal-oriented exchanges, revealing a widening mismatch with menu- and form-centric workflows [8, 36, 49]. In the following,

we review research that has tried to overcome these limitations by capitalizing on AI technology.

2.1 Conversational and Voice Interfaces

In response to the rigidity of GUIs, Conversational User Interfaces (CUIs) have emerged as a promising alternative to replace pointing and clicking with natural language (NL) commands and offer a more intuitive and direct interaction. Many of the current architectures for task-oriented dialogue systems, however, heavily rely on intent-and-slot-filling mechanisms [13]: the system is trained to recognize a limited set of user intents (e.g., 'BookFlight') and extract specific parameters, or "slots" (e.g., 'destination', 'date'), from the user utterance. Despite their success in simple domains, these systems exhibit significant brittleness when confronted with complex, multi-turn interactions. Their reliance on predefined schemas makes them unable to handle out-of-scope requests gracefully, and they often fail to maintain context over longer conversations [21]. Furthermore, the non-persistent nature of voice interaction makes it inadequate for tasks requiring the exploration or comparison of complex information, where visual interfaces provide essential affordances and a stable frame of reference [2, 27]. While recent advances in LLMs have drastically improved NL understanding, the fundamental limitations of a purely conversational modality for certain tasks remain [30, 43].

2.2 Component-Based and Declarative Web UI Development

The web frontend development has undergone a paradigm shift from imperative DOM manipulation (e.g., with jQuery) to a declarative, component-based model [10]. Frameworks like React [11] and Next.js [47] allow developers to define the UI as a function of its state while ensuring its rendering is consistent with the current state; this simplifies development and improves maintainability. Their component-based architecture has created a fertile ground for treating UI elements as machine-readable entities [22] and model-based techniques able to turn UI components into artifacts that can be automatically interpreted and adapted [1, 3]. In line with these works and recent efforts on generative, malleable Web UI interfaces [7, 28, 49], our work aims to extend the notion of machine-readability from model-based adaptation to real-time orchestration by an LLM.

2.3 LLMs for Web UI Orchestration

LLMs have opened new opportunities for mediating human–computer interaction, with their role of interpreters to map NL onto the activation of system computations. Research in this area has explored more flexible NL interfaces for web navigation [37]. A notable example is FLIN, a system that maps user commands to concept-level actions (e.g., "search for hotels") rather than low-level UI interactions (e.g., "click search button"), thus adapting easily across different websites [26]. This approach demonstrates the potential of grounding language in the affordances of an existing interface.

AI-driven UI generation is also rapidly evolving to create UIs directly from high-level textual descriptions and accelerate design and development. Recent studies have explored how UX practitioners would adopt such tools, highlighting their potential and the

existing gaps between model capabilities and user needs [8]. While these approaches are promising, they either focus on interpreting actions on existing, non-instrumented websites (like FLIN) or on generating new, static UIs from scratch (like GenUI). Other works, e.g., [15], instead explore the use of LLMs to dynamically generate context-sensitive GUIs during a conversation and show that such adaptive GUIs can improve usability in multi-turn interactions. A complementary line of work investigates how existing UI patterns themselves can be made “malleable”, giving end-users direct control over what information is displayed and how it is structured. Min et al. [28] also demonstrate that such malleability, especially when combined with AI assistance, enables users to align interfaces with their own decision-making criteria, thus pointing toward a future of more adaptive, user-driven interaction patterns.

Our framework also aims to improve UI adaptivity and flexibility, but differs by orchestrating the application’s own declarative components in real time, thus (i) preserving trusted logic (validation, security, A11y) from existing widgets, (ii) bounding the action space to documented props/events, and (iii) enabling testable, reproducible behaviors, thereby mitigating hallucinations and layout volatility typical of fully generative approaches.

2.4 The Identified Gap

The existing literature reveals a clear tension. On one hand, traditional graphical interfaces, while providing clear visual affordances, can become overly complex and inadequate for certain usage situations. Attempts to simplify them, by hiding or making elements transient, can negatively impact usability and performance [29], and challenge the user’s sense of control, even when they improve immersion [35]. Ephemeral adaptation draws attention to likely target GUI elements without altering their spatial layout [12]. While this can reduce visual search time, it still operates within a point-and-click paradigm, and its effectiveness depends heavily on prediction accuracy [4]. On the other hand, conversational and early-stage LLM-driven interfaces offer flexibility but lack the persistent, high-bandwidth feedback of a GUI, which is crucial for complex tasks.

This analysis exposes the lack of a framework that unifies the strengths of these diverse paradigms. Specifically, there is no integrated solution that combines, with experimental evidence:

- **A blending of verbal and visual interaction**, where NL is used to summon and control precise, persistent GUI components, merging the expressiveness of language with the clarity of a traditional visual interface.
- **A declarative, component-based UI definition** that leverages modern web frameworks for power and scale while preserving the trusted logic of existing widgets.
- **Automatic interpretability of these components by an LLM**, allowing a model to understand their function and required inputs directly from their definition, without needing to be trained on specific websites.

This paper tries to fill this gap by providing a novel architecture that bridges the world of declarative components with that of language-driven orchestration. and by investigating the effectiveness and efficiency of the resulting interaction paradigm.

3 Mirage: Models and Architecture

Mirage operationalizes this vision through a layered architecture that transforms NL requests into controlled invocations of application-level functionalities. A runtime agent leverages tool-calling to orchestrate functionalities encapsulated as “components”. The decision-making logic, delegated to the LLM, is separated from the execution logic defined by the components. This results in an extensible system whose capabilities can be expanded by defining new components, without altering the underlying orchestration engine.

3.1 Guiding Principles

Declarativity. Developers expose components to the application logic by declaring *what they do* (via an NL description) and *what data* they require (via a formal schema), rather than imperatively coding the logic of how and when they should be invoked. The framework’s LLM agent is responsible for interpreting user intents and orchestrating the appropriate components.

Reusability and Encapsulation. Each component defines the decision-time contract the agent relies on: a tool name, a semantic description, an optional typed parametersSchema, and a React view that emits structured UI events. Business logic and data access stay in the host application, so the same definition can be reused across different flows while Mirage handles selection and wiring.

LLM Interpretability. Components are designed from the ground up to be machine-readable and interpretable by an LLM. The combination of an NL description and a typed parameter schema forms a formal contract that enables the LLM to perform tool selection and parameter extraction from the user utterance.

Seamless Blending of Modalities. The Mirage workflow starts with a high-level NL command and results in the rendering of a precise, interactive GUI component. In a human-in-the-loop cycle, the user can thus refine or complete the expressed goal through familiar GUI affordances, blending conversational and GUI-based direct-manipulation paradigms.

Developer Effort. Mirage leverages existing React components, requiring only lightweight declarative descriptors to enable natural-language orchestration, without introducing new control logic or navigation structures.

3.2 Component Model

The central abstraction in Mirage is the **component**. It exposes the minimal contract for the LLM agent to choose the right tool, keeping the business logic within the host application. A component is an object with the following attributes (see Listing 1):

- **name:** A unique identifier used by the agent when selecting the component (e.g., “listResources” in Listing 1).
- **description:** An NL string that explains the component’s purpose, capabilities, and the situations in which it should be invoked (e.g., “List filtered university facilities such as classrooms, labs, or meeting rooms.”).
- **parametersSchema:** A schema (typically defined with Zod) that specifies the input parameters accepted by the component, including their types and validation rules (e.g., a category constrained to “CLASSROOM” or “LAB”, and an optional query string for keywords).

- **ui:** A React component (the *view*) responsible for rendering the interface once the host application has gathered any required data. The view can emit structured events (e.g., the “Update results” button in Listing 1 triggers an `onUpdate` event to refresh the list).

```

1  import { z } from 'zod';
2  import type { ComponentDefinition } from '@mirage-ai/core';
3
4  type ListResourcesProps = {
5    resources: Array<{ id: string; name: string; location: string }>;
6    onUpdate?: (event: {
7      component: string;
8      event: string;
9      payload: Record<string, unknown>;
10     timestamp: string;
11   }) => void;
12 };
13
14 export const listResources: ComponentDefinition<ListResourcesProps> = {
15   name: 'listResources',
16   description: 'List and filter university facilities such as classrooms,
17   ↪ labs, or meeting rooms.',
18   parametersSchema: z.object({
19     category: z.enum(['CLASSROOM', 'LAB']).optional(),
20     query: z
21       .string()
22       .optional()
23       .describe('Keywords to refine the search, e.g., "projector" or
24       ↪ "wheelchair accessible."'),
25   }),
26   ui: function ListResourcesView({ resources, onUpdate }) {
27     return (
28       <section>
29         <h3>Available Facilities</h3>
30         <ul>
31           {resources.map(({ id, name, location }) => (
32             <li key={id}>
33               <strong>{name}</strong> - {location}
34             </li>
35           ))}
36         </ul>
37         <button
38           onClick={() =>
39             onUpdate?.({
40               component: 'listResources',
41               event: 'refresh_requested',
42               payload: {},
43               timestamp: new Date().toISOString(),
44             })
45           >
46           Update results
47         </button>
48       </section>
49     );
50   };

```

Listing 1: Simplified definition of a Mirage component in a Next.js application for resource booking in a university department.

3.3 Interaction Workflow

The interaction model of Mirage is based on an iterative cycle that transitions from a user’s NL utterance to a rendered GUI. As summarized in Figure 1, the key stages are as follows:²

- (1) **Verbal Input:** The user initiates an interaction with an NL prompt. This prompt, along with the conversation history and any relevant UI events from prior interactions, is sent to the server-side Mirage agent.

- (2) **LLM Interpretation & Component Selection:** A first agent, called *Mirage Decision Agent* (MDA), which is bound to the application’s available components, interprets the user’s intent and selects the most appropriate component, extracting from the prompt the values to instantiate its parameters. Parameters can be either mandatory or optional, as specified in the component’s schema. If required parameters are missing, the component cannot be mounted directly; a second agent, called *Mirage Response Agent* (MRA), guides the user to supply the missing values (e.g., through *clarifying questions*) Optional parameters, by contrast, allow the component to be invoked with partial data, letting the user complete or refine the input through the rendered UI rather than specifying everything in natural language. This separation of roles reduces prompt entanglement and improves interpretability and modularity.
- (3) **Component Mounting, Interaction & Feedback:** Once the agent returns a component decision, the host application prepares the props, and mounts the component UI. Users then interact with the rendered UI, producing structured `uiEvents` and, when applicable, component-driven updates; both are fed back to Mirage to enrich context for disambiguation, multi-step tasks, and adaptive guidance.

3.4 Technical Architecture

Built on the Next.js declarative framework³, Mirage is organized into a layered architecture that separates conversational reasoning, component orchestration, and UI presentation, and operates a two-stage agent loop (selection and guidance), as illustrated in Figure 2.

Verbal Layer. The host application captures the user utterance and maintains conversation history and recent UI events in its own state. On each turn, it sends this context (and a session identifier) to the Mirage agent runtime, which runs on the server to keep model API keys secure. By default, the runtime is stateless: it consumes the provided history/events and, when present, the latest `AgentDecision`; any persistence of interaction context is the responsibility of the host application or its server.

Component Orchestration Layer. At the core lies the *component contract*, which defines each interactive unit through a stable name, an NL description, an optional Zod parameter schema, and an optional React UI. A *Component Registry* keeps all definitions consistent and deduplicated, while a builder translates them into structured tools with strict or loose schema validation depending on available metadata. The agent loop runs in two stages: the MDA selects the component and extracts its parameters; the MRA produces the assistant utterance that frames the decision and may suggest follow-up interfaces. Once a component is selected, the runtime returns an `AgentDecision` (component name + validated parameters). The host React application then mounts the component and supplies any domain data; domain-specific business logic remains encapsulated within the component, outside Mirage’s scope.

UI Layer. At the client side, the host React application mounts the component specified by the `AgentDecision`, injecting validated

²A video demonstration is available at this link: <https://tinyurl.com/Mirage-DemoVideo>

³The current prototype is available at this link: <https://tinyurl.com/Mirage-Framework>

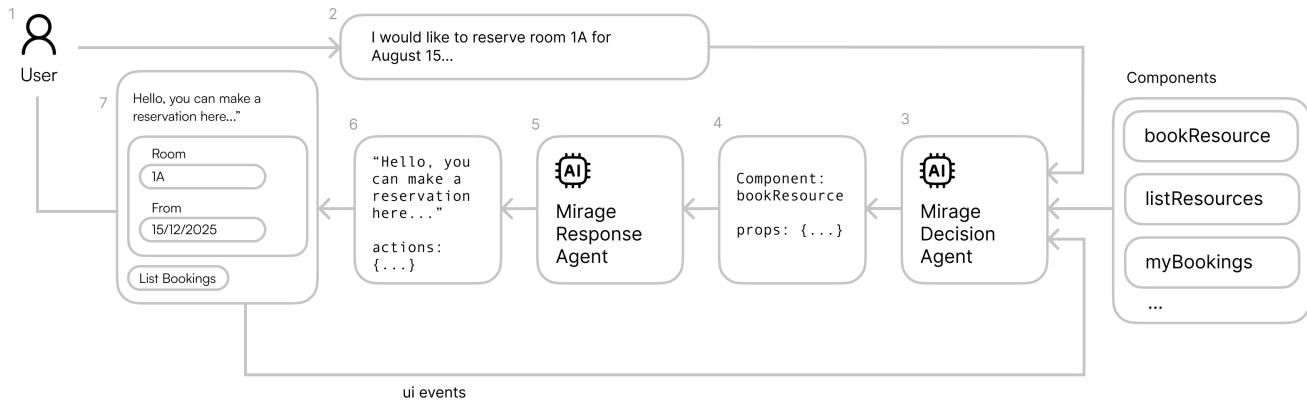


Figure 1: Interaction workflow in Mirage.

props and any required domain data. User interactions (e.g., form submissions, button presses) are captured as structured `uiEvents`; components may also emit *updates* via an `onUpdate` mechanism. Both `uiEvents` and *updates* are persisted in the application state and included in subsequent prompts, enabling multi-step flows and adaptive agent behavior.

Overall, Mirage enforces a deliberate separation of concerns: LLMs handle intent interpretation and guidance; Mirage ensures safe orchestration and schema validation; and the host application remains responsible for data retrieval, side effects, and view logic. This design allows new interactive flows to be introduced declaratively by defining components, without modifying the agent loop or entangling conversational AI with core business code.

3.5 Application Example

To illustrate the framework in action, we consider the **facility booking system** offered by our university. This use case was implemented and used in our experimental setup (see Sections 4 and 5). Three components are defined: `listResources(category, query)`, enabling finding and displaying facilities (see Listing 1); `bookResource(resourceId, date, ...)`, to create a reservation; `myBookings()`, to display the current user's reservations.

As illustrated in Figure 1, a prompt like "Are there any classrooms available with a projector?" is mapped by the LLM to `listResources(category: 'CLASSROOM', query: 'projector')`, which then renders a list of matching rooms. A subsequent command, e.g., "Book the Alpha room tomorrow from 10 to 11", would trigger the `bookResource` component. If any parameters are missing (e.g., the full date), the component can be invoked with partial or null parameters, and the rendered UI facilitates completion by the user. As shown in Figure 3, the rendered UI is then a form pre-filled with the known information (room name, times) and prompts the user to fill in the missing date, following a human-in-the-loop workflow for task completion. The same pattern would apply to other domains (e.g., e-commerce workflows, dashboard operations, or educational content navigation) by adding or composing components without modifying the agent loop.

4 User Study

To assess the usability of the Mirage Web navigation paradigm, we conducted a controlled user study comparing Mirage performance for the case study illustrated in Section 3.5 with navigation in the original intranet system. We tested the following hypotheses:

- **H1 (Efficiency):** Mirage reduces Task Completion Time (TCT) compared to the traditional intranet interface.
- **H2 (Accuracy):** Mirage leads to fewer errors and more accurate task completion.
- **H3 (Cognitive Load):** Mirage lowers subjective workload, measured with the NASA-TLX global score.
- **H4 (Usability):** Mirage achieves higher usability ratings, measured with the System Usability Scale (SUS).

4.1 Experimental Design

We adopted a within-subjects design with two conditions: *Mirage* and the university intranet (*Intranet*). A total of 15 participants (age ≥ 18 , digitally literate, 8 self-identified as females and 7 as males) were recruited: 10 aged 18–24, 1 aged 25–34, and 4 aged 55–64.

Each participant completed booking tasks under both conditions (Mirage vs. Intranet) with equivalent business rules and terminology. Order of exposure was counterbalanced using a Latin square (AB/BA) to mitigate learning and memory effects.

4.2 Procedure

The study collected data for both *objective* metrics (TCT in seconds, and task accuracy as a binary checklist of correct booking attributes), and *subjective* metrics (NASA-TLX, SUS, and qualitative feedback from spontaneous observations and interviews).

Each study session lasted about 40–50 minutes. After signing the consent form and completing a short demographic questionnaire, the participants received a five-minute guided introduction to both systems, during which they tried two example actions per interface, distinct from the study tasks. Participants next performed a booking task with the first interface (order randomized), followed by the completion of the NASA-TLX and SUS questionnaires. After a short break, they repeated the task with the second interface and filled

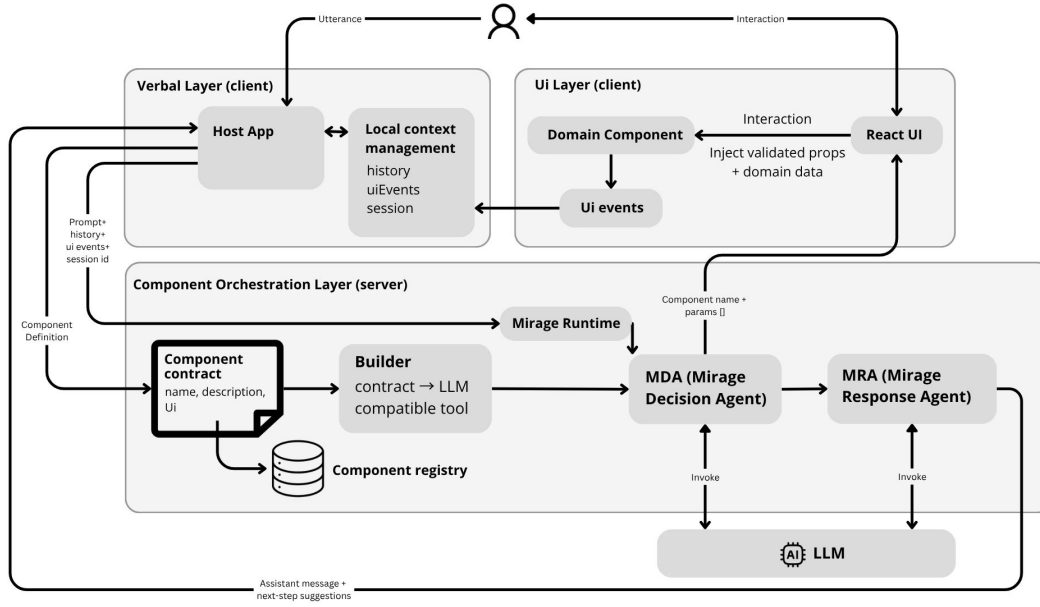


Figure 2: Layered technical architecture of the Mirage framework.

out the same questionnaires. The session concluded with a semi-structured interview exploring perceptions, breakdowns, trust, and preferred contexts of use.

4.3 Results

4.3.1 Objective User Performance. For the *User Efficiency* (H1), we first compared task completion time (TCT) between Mirage and Intranet. On average, Mirage enabled participants to complete tasks more quickly (mean = 204.6s, $SD = 126.1s$) than the Intranet (mean = 296.1s, $SD = 152.6$). The differences were normally distributed (Shapiro–Wilk $W = 0.97$, $p = 0.87$), and both a paired t-test ($t(14) = 3.38$, $p = 0.004$) and a Wilcoxon signed-rank test ($W = 9.5$, $p = 0.007$) confirmed the effect as statistically significant (see Figure 3.a). For the *User Accuracy* (H2), each participant was required to complete a booking workflow consisting of three steps: finding a suitable room, submitting the booking, and verifying the reservation. Out of 15 participants, 14 successfully completed the workflow with Mirage; one participant, however, booked the wrong room, leading to an incorrect outcome. In the Intranet condition, 14 out of 15 participants completed the workflow, while one participant failed to finish within the 10-minute limit. Thus, accuracy was comparable across conditions (93.3%), but Mirage achieved significantly faster execution without increasing error rates.

4.3.2 Cognitive Load and Perceived Usability. *Subjective Workload* (H3) was measured with the NASA-TLX, and usability with the SUS. Mirage consistently yielded lower workload and higher usability scores (see Figure 3). The average NASA-TLX score was 37.9 ($SD = 11.0$) for Mirage compared to 51.3 ($SD = 16.3$) for the Intranet. A paired t-test showed this difference to be statistically significant ($t(14) = 3.76$, $p = 0.002$, Cohen’s $d = 0.97$). On the *Perceived Usability* side (H4), the SUS score for Mirage averaged 79.3 ($SD = 10.3$), which is considered “good” usability,

whereas the Intranet scored only 47.3 ($SD = 17.4$), typically interpreted as “poor” usability. The difference was highly significant ($t(14) = -5.81$, $p < 0.001$, Cohen’s $d = 1.50$) (see Figure 3.b).

A dimension-level analysis of NASA-TLX provided further insights. For *mental effort*, participants reported lower demands with Mirage (mean = 36.0) compared to the Intranet (mean = 56.0), a significant difference ($t(14) = 2.50$, $p = 0.025$). *Physical effort* was also significantly reduced with Mirage (mean = 9.3) relative to the Intranet (mean = 22.7; $t(14) = 2.17$, $p = 0.048$). No significant differences emerged for *time load* (Mirage = 35.3, Intranet = 39.3; $t(14) = 0.81$, $p = 0.433$) or self-rated *performance* (Mirage = 91.3, Intranet = 83.3; $t(14) = -1.23$, $p = 0.238$). However, Mirage required substantially less *commitment* (mean = 34.7) compared to the Intranet (mean = 62.7; $t(14) = 3.70$, $p = 0.002$), and induced lower *frustration* (Mirage = 20.7, Intranet = 44.0; $t(14) = 3.46$, $p = 0.004$). Mirage was thus perceived as less demanding and less frustrating, maintaining comparable perceptions of time pressure and task success.

4.3.3 Qualitative Analysis. Semi-structured interviews provided insights that complemented the quantitative results. A thematic analysis revealed the three following clusters of recurring themes.

Perceived Advantages. Participants consistently described Mirage as more *immediate*, *intuitive*, and *less effortful* than the institutional Intranet. Several highlighted that typing a request (“*writing is easier than finding it in menus*”) was faster than navigating nested structures. Mirage was also appreciated for its ability to understand requests even when expressed imprecisely (“*it understands even if I don’t know the exact terms*”), and for reducing cognitive load by presenting only the relevant component instead of multiple menus. For complex or infrequent operations, participants valued the system’s guidance and sequential structure. Some also suggested that integrating speech input would further increase efficiency.

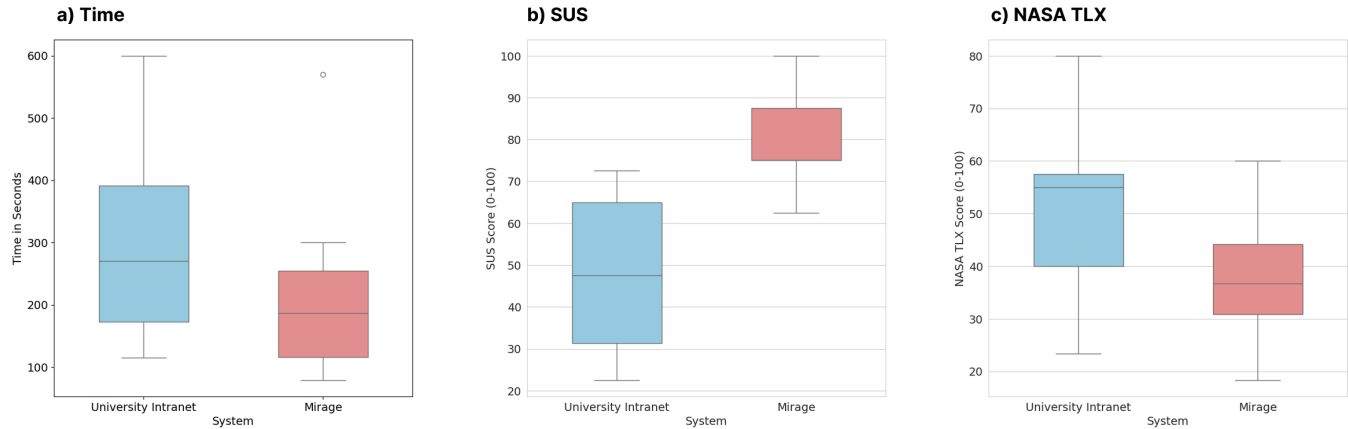


Figure 3: Comparison of Times, NASA-TLX workload, and SUS usability scores for Mirage and Intranet.

Breakdowns and Limitations. Despite its benefits, Mirage was not perceived as universally superior. A few participants noted that when one already knows the exact navigation path, menus can be faster. Others reported concerns about over-reliance on NL input: uncertainty about how to phrase queries, lack of transparency, or fear that typos might prevent task completion. In Mirage, autocompletion and confirmation mechanisms were seen as essential: one participant explicitly requested more robust confirmations (“it has to ask before doing everything”) to avoid booking errors. On the Intranet side, breakdowns were primarily due to *poor information architecture*: lack of filtering options, inconsistent terminology, and difficulty locating functionality. However, one participant perceived the Intranet as more *intuitive* simply because it displayed the full set of options, aligning with familiar conventions.

Use Cases and Trust. Trust emerged as a key factor in acceptability. Several participants expressed partial reliance on Mirage: while they found it useful and efficient, they reported that they would still “double-check” the output or worry about making mistakes. Others stressed that trust would grow with use and with visible confirmation steps. Interestingly, some envisioned *hybrid interfaces*, combining Mirage’s conversational guidance with traditional menu-based navigation (“if the two were integrated together, it would be perfect”). The chat-based paradigm was seen as especially suited for complex or rarely used tasks (e.g., facility booking, academic plan inquiries), while frequent or simple tasks might remain better served by structured GUIs. Participants also emphasized personalization as a trust-building factor: they would like Mirage to remember past actions and preferences (“it should know my recent bookings”). Finally, concerns were raised about accessibility: while some found Mirage much easier, others cautioned that older or less tech-savvy users might initially struggle with a purely conversational interface.

5 Technical Validation of Component Mapping

To complement the user study, we evaluated the Mirage’s ability to map NL prompts to intranet components. We reconstructed the menu structure of the institutional intranet, deriving a set of representative Mirage components annotated with NL descriptions

and parameter schemas. Starting from the menu structure and using ChatGPT, we generated 50 prompts per component covering both operational tasks and informational queries. We then assessed Mirage’s ability to select the correct component via *precision*, *recall*, and *F1-score*; *execution time* was recorded as well⁴.

5.1 Results

With GPT-5 as the decision agent, Mirage achieved a macro-averaged F1 of 75.6%, while GPT-5-nano reached 68.9%. Execution times averaged 13.3s with GPT-5 (min 3.6s, max 83.7s) and 6.4s with GPT-5-nano (min 2.5s, max 71.7s), highlighting a trade-off between accuracy and latency. GPT-5 achieved a macro-average precision of 85.7%, recall of 73.2%, and F1 = 75.6%. The micro-average F1 is 75.3%, while the weighted-average F1, which accounts for class imbalance, reaches 77.7%. Bootstrap-estimated 95% confidence intervals indicate accuracy = 73.2–77.2% and macro F1 = 68.3–82.9%. A detailed per-class confusion matrix, highlighting major sources of semantic overlap (e.g., between regulations and howToGuides), is provided in Figure 4 in the Appendix A.

Accuracy varied across components. Operational elements (e.g., bookResource, requestTransportDocument), reached near-perfect accuracy, while other task-oriented components like listResources obtained high recall but lower precision, reflecting ambiguity with semantically related functions (F1=87.7). Informational components with distinctive semantics, such as organizationalChart, also performed very well (F1≈99.0). By contrast, documentation-oriented categories with generic or overlapping descriptions (e.g., regulations, howToGuides, facultyInfo) showed lower scores, in some cases below 30% F1. These results highlight the centrality of semantic separability: components with distinctive descriptions are classified reliably, while broad or overlapping terminology attracts false positives. Retrieval-augmented approaches could provide a more robust solution by grounding queries in component content rather than metadata alone.

⁴Test material and detailed results are available at this link.: <https://tinyurl.com/mirageTechnicalValidation>

6 Discussion

Our user study shows that *Mirage* significantly reduced task completion time compared to the traditional intranet system, without compromising accuracy. This improvement can be attributed to the reduction in menu navigation and the ability to issue requests in natural language, which led to a more direct and efficient interaction flow. These findings are consistent with prior work showing that conversational interfaces can reduce learning effort and enhance user experience, [23, 39, 42, 45]. Nevertheless, trade-offs emerged. While *Mirage* improved efficiency in user performance, it also introduced occasional errors in NL understanding and higher latency due to LLM processing. Component mapping accuracy was competitive but coupled with a measurable latency gap. Crucially, accuracy seems to depend more on the semantic separability of component metadata than on the nature (operational vs. informational) of the task: components with discriminative descriptions/parameter schemas were recognized near-perfectly, whereas broad or overlapping labels attracted false positives. This suggests that metadata quality is a first-order design variable for LLM-driven orchestration, while model size mainly trades accuracy for responsiveness.

During the user study, some participants reported that GUIs remained more convenient in scenarios where they already knew the exact navigation path, consistent with the findings of Liu et al. [23] that show how GUIs may benefit users with a rational decision style and repeated use. This suggests that *Mirage* complements, rather than replaces, traditional GUIs by offering a more accessible entry point for less structured or multi-step tasks.

6.1 Implications for Design

Our findings confirm that conversational systems improve usability, but require *hybrid support mechanisms*. The study emphasized the potential of combining conversational interfaces with graphical navigation. This recalls prior work highlighting that voice or text-based interaction, while intuitive, increases mental effort if not supported by clarifying mechanisms [14]. A promising approach is the *Mirage declarative component model*, which offers the benefits of conversational interaction while preserving the precision of graphical manipulation [24]. Such hybrid designs appear especially suited to domains with complex rules or multi-step workflows.

On the component-specification side, our results indicate that clear, discriminative, and schema-rich component descriptions are pivotal for robust orchestration [9]. It is important to adopt controlled vocabularies, avoid overloaded synonyms, and provide typed props with examples. When semantic overlap is unavoidable, *top-k* selection with confirmation or RAG-grounded disambiguation could be adopted. This spec-driven hygiene mitigates false positives without sacrificing the benefits of a conversational entry point. Analogously to how “prompt engineering” emerged for LLMs, we argue for *spec engineering* of UI component, even if written in natural language, as a first-class practice for reliable LLM-driven orchestration.

Trust remains critical. Users’ feedback revealed skepticism about delegating full control to the AI, in line with Schilke and Reimann [38], who demonstrate that transparency about AI usage can paradoxically reduce trust. Hence, conversational interfaces must provide validation steps and opportunities for user oversight, rather than relying solely on automated agent reasoning [36].

Recent tools such as Cursor, Lovable, or Claude Code primarily support developer-side code generation and prototyping. *Mirage* instead operates at runtime, focusing on the interpretation and orchestration of existing, developer-defined UI components; these approaches are therefore complementary rather than overlapping.

6.2 Limitations and Future Research

Our experimental setting has limitations. The user sample size ($N=15$) limits the usability findings’ generalizability. Then, the domain specificity may bias results towards systems where structured queries are straightforward, limiting external validity [44]. However, the goal of this evaluation is not to benchmark intent-classification models, but to assess the feasibility and robustness of LLM-driven component orchestration.

The scalability of *Mirage* should be evaluated in other domains, such as customer relationship management, e-commerce, and health-care, where reducing cognitive load may bring substantial benefits [42, 45]. Longitudinal studies are needed to assess long-term acceptability and trust. Prior work suggests that familiarity with chatbots reduces initial skepticism [44]. Further exploration of multimodal designs (e.g., combining text, voice, and GUI interaction) could maximize efficiency and transparency [14]. Designers/developers’ acceptability also needs investigation [8].

On the technical side, the accuracy evaluation used LLM-generated prompts, which may not fully capture real-world phrasing and could misestimate performance. A follow-up with human-authored and in-the-wild logs (plus inter-rater adjudication of “correct component”) would strengthen external validity.

Component matching optimization is necessary to reduce latency and semantic errors. Alternative component selection strategies (e.g., RAG over website content) should be evaluated for low-separability categories to improve macro-F1 while maintaining UX-acceptable median latency [16, 17].

7 Conclusions

Our research argues for a shift from menu-driven design to capabilities with contracts. End-to-end UI generation by LLMs represents a promising research direction; however, our work shows that exposing UI functionality as declared, machine-readable components enables on-demand, goal-driven composition that is fast for users and dependable for developers. The longer-term vision, aligned with current trends [32], is a commons of interoperable components, discoverable and composable, where LLMs act as orchestrators. Achieving it requires advancing standards for component specifications, methods for trust, testing, and governance, and hybrid multimodal patterns that keep users in control. In this light, *Mirage* is not an end-product but a design substrate for an *operating system of the Web*: capability-centric, auditable, and inclusive.

Acknowledgments

This research is partially funded by Next Generation EU, NRRP Initiative, Mission 4, Component 2, Investment 1.3, D.D. MUR 341, 15.03.2022 (PE0000013 – “Future Artificial Intelligence Research – FAIR” - CUP: H97G22000210007). We gratefully acknowledge Google for the Society-Centered AI Award supporting this research.

References

- [1] Silvia Abrahão, Emilio Insfran, Arthur Sluÿters, and Jean Vanderdonckt. 2021. Model-based intelligent user interface adaptation: challenges and future directions. *Softw. Syst. Model.* 20, 5 (Oct. 2021), 1335–1349. doi:10.1007/s10270-021-00909-7
- [2] Gonul Ayci, Murat Sensoy, Arzucan Özgür, and Pinar Yolum. 2023. Uncertainty-Aware Personal Assistant for Making Personalized Privacy Decisions. *ACM Trans. Internet Technol.* 23, 1, Article 13 (March 2023), 24 pages. doi:10.1145/3561820
- [3] Carlo Bernaschina, Sara Comai, and Piero Fraternali. 2018. Formal semantics of OMG's Interaction Flow Modeling Language (IFML) for mobile and rich-client application model driven development. *J. Syst. Softw.* 137 (2018), 239–260. doi:10.1016/j.jss.2017.11.067
- [4] Sara Bouzit, Denis Chêne, and Gaelle Calvary. 2014. From appearing to disappearing ephemeral adaptation for small screens. In *Proceedings of the 26th Australian Computer-Human Interaction Conference on Designing Futures: the Future of Design*. ACM, Sydney New South Wales Australia, 41–48. doi:10.1145/2686612.2686619
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, and et al. 2020. Language Models are Few-Shot Learners. doi:10.48550/arXiv.2005.14165 [cs].
- [6] Cristina Liviana Caldiroli, Francesca Gasparini, Silvia Corchs, Andrea Mangia-tordi, Roberta Garbo, Alessandro Antonietti, and Fabrizia Mantovani. 2023. Comparing online cognitive load on mobile versus PC-based devices. *Personal and Ubiquitous Computing* 27, 2 (apr 2023), 495–505. doi:10.1007/s00779-022-01707-8
- [7] Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proceedings of CHI 2025 (CHI '25)*. ACM, Article 686, 20 pages. doi:10.1145/3706598.3713285
- [8] Xiang 'Anthony' Chen, Tiffany Knearear, and Yang Li. 2025. The GenUI Study: Exploring the Design of Generative UI Tools to Support UX Practitioners and Beyond. In *Proceedings of the 2025 ACM Designing Interactive Systems Conference*. 1179–1196. doi:10.1145/3715336.3735780 arXiv:2501.13145 [cs].
- [9] Michael Coblenz, Wentao Guo, Kamatchi Voozhian, and Jeffrey S. Foster. 2023. A Qualitative Study of REST API Design and Specification Practices. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 148–156. doi:10.1109/VL-HCC57772.2023.00025
- [10] Conal Elliott and Paul Hudak. 1997. Functional reactive animation. *SIGPLAN Not.* 32, 8 (Aug. 1997), 263–273. doi:10.1145/258949.258973
- [11] Facebook. 2013. React: A JavaScript library for building user interfaces. <https://reactjs.org>. Accessed: September 9, 2025.
- [12] Leah Findlater, Karyn Moffatt, Joanna McGrenere, and Jessica Dawson. 2009. Ephemeral adaptation: the use of gradual onset to improve menu selection performance. In *Proceedings of CHI 2009*. ACM, Boston MA USA, 1655–1664. doi:10.1145/1518701.1518956
- [13] Chih-Wen Goo, Guang Gao, Yun-Kai Hsu, Chih-Li Huo, Tsung-Chieh Chen, Keng-Wei Hsu, and Yun-Nung Chen. 2018. Slot-Gated Modeling for Joint Slot Filling and Intent Prediction. In *Proceedings of the 2018 Conference of the North American Chapter of the ACL: Human Language Technologies, Volume 2 (Short Papers)*. ACL, New Orleans, Louisiana, 753–757. doi:10.18653/v1/N18-2118
- [14] Florian Habler, Marco Peisker, and Niels Henze. 2019. Differences between smart speakers and graphical user interfaces for music search considering gender effects. In *Proceedings of the 18th International Conference on Mobile and Ubiquitous Multimedia*. ACM, Pisa Italy, 1–7. doi:10.1145/3365610.3365627
- [15] Nobukatsu Hojo, Kazutoshi Shinoda, Yoshihiro Yamazaki, Keita Suzuki, Hiroaki Sugiyama, Kyosuke Nishida, and Kuniko Saito. 2025. GenerativeGUI: Dynamic GUI Generation Leveraging LLMs for Enhanced User Interaction on Chat Interfaces. (2025).
- [16] Pengcheng Jiang, Siru Ouyang, Yizhuo Jiao, Ming Zhong, Runchu Tian, and Jiawei Han. 2025. Retrieval And Structuring Augmented Generation with Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (Toronto ON, Canada) (KDD '25)*. ACM, 6032–6042. doi:10.1145/3711896.3736557
- [17] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *ACM Trans. Comput. Syst.* (Sept. 2025). doi:10.1145/3768628 Just Accepted.
- [18] Junhan Kong, Jacob O. Wobbrock, Tianyuan Cai, and Zoya Bylinskii. 2025. Supporting Mobile Reading While Walking with Automatic and Customized Font Size Adaptations. In *Proceedings of CHI 2025 (CHI '25)*. ACM, Article 1007, 13 pages. doi:10.1145/3706598.3713367
- [19] Steve Krug and Roger Black. 2000. *Don't Make Me Think! A Common Sense Approach to Web Usability* (1st ed.). Que Corp., USA.
- [20] Karel Kubicek, Jakob Merane, Ahmed Bouhoula, and David Basin. 2024. Automating Website Registration for Studying GDPR Compliance. In *Proceedings of the ACM Web Conference 2024 (Singapore, Singapore) (WWW '24)*. ACM, 1295–1306. doi:10.1145/3589334.3645709
- [21] Stefan Larson, Anish Mahendran, Joseph J. Peper, and al. 2019. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of EMNLP-IJCNLP*. ACL, Hong Kong, China, 1311–1316. doi:10.18653/v1/D19-1131
- [22] Grzegorz Lehmann, Marco Blumendorf, and Sahin Albayrak. 2010. Development of context-adaptive applications on the basis of runtime user interface models. In *Proceedings of the 2nd ACM SIGCHI symposium on Engineering interactive computing systems*. ACM, Berlin Germany, 309–314. doi:10.1145/1822018.1822068
- [23] Xuanhui Liu, Tim Rietz, and Alexander Maedche. 2024. Conversational versus graphical user interfaces: the influence of rational decision style when individuals perform decision-making tasks repeatedly. *Universal Access in the Information Society* (June 2024). doi:10.1007/s10209-024-01122-1
- [24] Andreas Lommatzsch, Brandon Llanque, Vinay Srinath Rosenberg, Syed Ali Murad Tahir, Hristo Dimitrov Boyadzhiev, and Maurice Walny. [n. d.]. Combining Information Retrieval and Large Language Models for a Chatbot that Generates Reliable, Natural-style Answers. ([n. d.]).
- [25] Damien Masson, Sylvain Malacria, Géry Casiez, and Daniel Vogel. 2024. Direct-GPT: A Direct Manipulation Interface to Interact with Large Language Models. In *Proceedings of CHI 2024 (Honolulu, HI, USA) (CHI '24)*. ACM, Article 975, 16 pages. doi:10.1145/3613904.3642462
- [26] Sahisnu Mazumder and Oriana Riva. 2021. FLIN: A Flexible Natural Language Interface for Web Navigation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. ACL, Online, 2777–2788. doi:10.18653/v1/2021.naacl-main.222
- [27] Raphael Menges, Steffen Staab, Christoph Schaefer, Tina Walber, and Chandan Kumar. 2025. What Did My Users Experience? Discovering Visual Stimuli on Graphical User Interfaces of the Web. *ACM Trans. Web* 19, 2, Article 11 (March 2025), 63 pages. doi:10.1145/3715881
- [28] Bryan Min, Allen Chen, Yining Cao, and Haijun Xia. 2025. Malleable Overview-Detail Interfaces. In *Proceedings of CHI 2025*. ACM, Yokohama Japan, 1–25. doi:10.1145/3706598.3714164
- [29] Mubashar Munir and Pietro Murano. 2023. The Usability of Hidden Functional Elements in Mobile User Interfaces. In *Proceedings of the 25th International Conference on Enterprise Information Systems*. SCITEPRESS - Science and Technology Publications, Prague, Czech Republic, 256–263. doi:10.5220/0011827700003467
- [30] Mohammad (Matt) Namvarpour and Afsaneh Razi. 2025. The Art of Talking Machines: A Comprehensive Literature Review of Conversational User Interfaces. In *Proceedings of the 7th ACM Conference on Conversational User Interfaces (CUI '25)*. ACM, Article 45, 18 pages. doi:10.1145/3719160.3736621
- [31] Jakob Nielsen. 2000. *Designing web usability*. New Riders Publ. <https://www.worldcat.org/oclc/43116529>
- [32] OpenAI. 2025. Introducing apps in ChatGPT and the new Apps SDK (Oct 6th 2025). <https://openai.com/index/introducing-apps-in-chatgpt/>.
- [33] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. doi:10.48550/arXiv.2203.02155 arXiv:2203.02155 [cs].
- [34] Fabio Paterno, Carmen Santoro, and Lucio Davide Spano. 2009. MARIA: A universal, declarative, multiple abstraction-level language for service-oriented applications in ubiquitous environments. *ACM Trans. Comput.-Hum. Interact.* 16, 4, Article 19 (Nov. 2009), 30 pages. doi:10.1145/1614390.1614394
- [35] Dale Patterson and Sean Costain. 2015. The Effectiveness of Transient User Interface Components. *User Interfaces* 162 (2015).
- [36] L. Piro, G. Di Fede, E. Pucci, S. Tolomeo, and M. Matera. 2025. Leveraging LLMs for Voice-based Form Filling on the Web: The ConWeb Approach. In *Web Engineering: 25th International Conference, ICWE 2025, Delft, The Netherlands, June 30–July 3, 2025, Proceedings (Lecture Notes in Computer Science, Vol. 15749)*. Himanshu Verma, Alessandro Bozzon, Andrea Mauri, and Jie Yang (Eds.). Springer, Cham.
- [37] Emanuele Pucci, Isabella Possaghi, Claudia Maria Cutrupi, Marcos Báez, Cinzia Cappiello, and Maristella Matera. 2023. Defining Patterns for a Conversational Web. In *Proceedings of CHI 2023*. ACM, 118:1–118:17. doi:10.1145/3544548.3581145
- [38] Oliver Schilke and Martin Reimann. 2025. The transparency dilemma: How AI disclosure erodes trust. *Organizational Behavior and Human Decision Processes* 188 (May 2025), 104405. doi:10.1016/j.obhdp.2025.104405
- [39] Johanna Schmidhuber, Stephan Schlögl, and Christian Ploder. 2021. Cognitive Load and Productivity Implications in Human-Chatbot Interaction. In *2021 IEEE 2nd International Conference on Human-Machine Systems (ICHMS)*. 1–6. doi:10.1109/ICHMS53169.2021.9582445 arXiv:2111.01400 [cs].
- [40] Li Shi, Nilavra Bhattacharya, Anubrata Das, and Jacek Gwizdzka. 2023. True or false? Cognitive load when reading COVID-19 news headlines: an eye-tracking study. In *Proceedings of the 2023 Conference on Human Information Interaction and Retrieval (Austin, TX, USA) (CHIIR '23)*. ACM, 107–116. doi:10.1145/3576840.3578290
- [41] B. Shneiderman. 1996. The eyes have it: a task by data type taxonomy for information visualizations. In *Proceedings 1996 IEEE Symposium on Visual Languages*. IEEE Comput. Soc. Press, Boulder, CO, USA, 336–343. doi:10.1109/VL.1996.545307
- [42] Hiral Soni, Julia Ivanova, Hattie Wilczewski, Alexandra Bailey, Triton Ong, Alexa Narma, Brian E. Bunnell, and Brandon M. Welch. 2022. Virtual conversational agents versus online forms: Patient experience and preferences for health data collection. *Frontiers in Digital Health* 4 (Oct. 2022), 954069. doi:10.3389/fdgth.

- 2022.954069
- [43] Sofia Eleni Spatharioti, David Rothschild, Daniel G Goldstein, and Jake M Hofman. 2025. Effects of LLM-based Search on Decision Making: Speed, Accuracy, and Overreliance. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM, Article 1025, 15 pages. doi:10.1145/3706598.3714082
 - [44] Wondwesen Tafesse and Yoseph Mamo. 2025. A comparison of conversational chatbots and the internet for consumer information search. *Behaviour & Information Technology* (June 2025), 1–18. doi:10.1080/0144929X.2025.2517215
 - [45] Jhonatan Jeferson Tapia-Hoyos. 2024. Chatbots in the service of university students: A review. *Social Sciences in Brief* 1 (Nov. 2024), 1–7. doi:10.47909/ssb.08
 - [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. doi:10.48550/arXiv.1706.03762 arXiv:1706.03762 [cs].
 - [47] Vercel. 2016. Next.js: The React Framework for the Web. <https://nextjs.org/>. Documentation, accessed 2025-10-03.
 - [48] Jason Wu, Siyan Wang, Siman Shen, Yi-Hao Peng, Jeffrey Nichols, and Jeffrey P Bigham. 2023. WebUI: A Dataset for Enhancing Visual UI Understanding with Web Semantics. In *Proceedings of CHI 2023* (Hamburg, Germany) (*CHI '23*). ACM, Article 286, 14 pages. doi:10.1145/3544548.3581158
 - [49] Jackie (Junrui) Yang, Yingtian Shi, Yuhan Zhang, Karina Li, Daniel Wan Rosli, Anisha Jain, Shuning Zhang, Tianshi Li, James A. Landay, and Monica S. Lam. 2024. ReactGenie: A Development Framework for Complex Multimodal Interactions Using Large Language Models. In *Proceedings of CHI 2024* (Honolulu, HI, USA). ACM, Article 483, 23 pages. doi:10.1145/3613904.3642517
 - [50] Jin Yu, Boualem Benatallah, Régis Saint-Paul, Fabio Casati, Florian Daniel, and Maristella Matera. 2007. A framework for rapid integration of presentation components. In *Proceedings of the 16th International Conference on World Wide Web, WWW 2007, Banff, Alberta, Canada, May 8-12, 2007*. ACM, 923–932. doi:10.1145/1242572.1242697

A Appendix

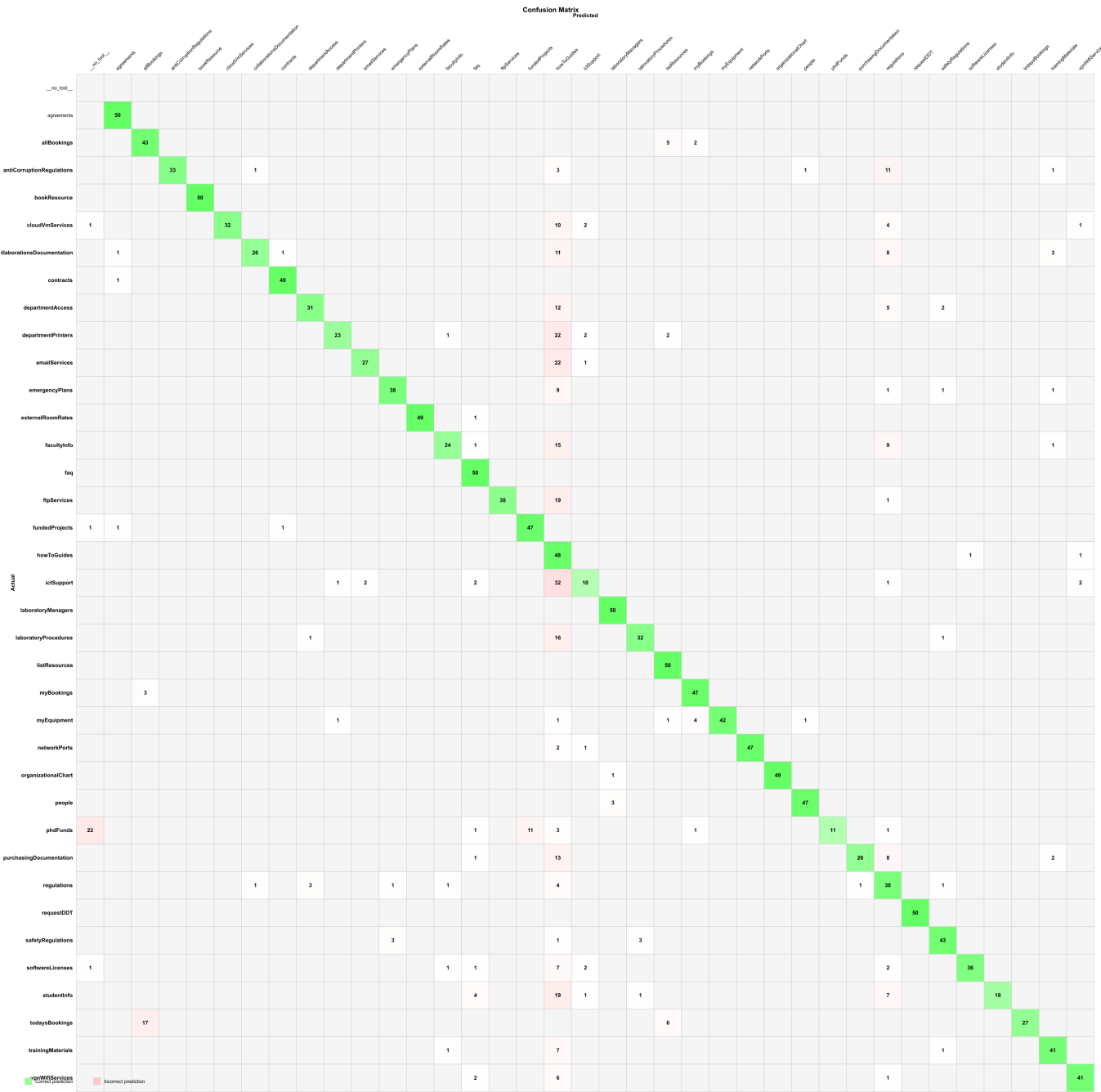


Figure 4: Per-class confusion matrix (rows: actual; columns: predicted). Darker cells indicate higher confusion